

Analisis Exploratorio de Datos (EDA)

Fundamentos de Aprendizaje Automático

Germán A. Montoya O.
gamontoyao@eafit.edu.co

Universidad EAFIT

Medellín 2026

Estadística Descriptiva

- Antes de aplicar modelos complejos, un científico de datos debe entender la naturaleza de sus datos.
- La estadística descriptiva permite:
 - **Resumir y visualizar:** Calcular medidas de tendencia central (media, mediana, moda), medidas de dispersión (varianza, desviación estándar, rango) y visualizaciones efectivas para tener una visión rápida y concisa de los datos.
 - **Identificar patrones y anomalías:** Graficar distribuciones, diagramas de dispersión, histogramas para visualizar la distribución de los datos, identificar valores atípicos (outliers) y patrones iniciales.



Fuente: <https://emeritus.org/in/learn/what-is-artificial-intelligence/>

Estadística Descriptiva

Herramientas y medidas para resumir y presentar los datos:

- **Medidas de Tendencia Central:** Indican el "centro" o el valor típico de un conjunto de datos.
 - **Media (Promedio):** La suma de todos los valores dividida por el número total de valores. Es sensible a los valores atípicos (outliers).
`media_matematicas = df['Matematicas'].mean()`
 - **Mediana:** El valor central en un conjunto de datos ordenado. Si hay un número par de datos, es el promedio de los dos valores centrales. Es robusta a los outliers.
`mediana_matematicas = df['Matematicas'].median()`
 - **Moda:** El valor que aparece con mayor frecuencia en un conjunto de datos. Útil para datos categóricos.
`moda_matematicas = df['Matematicas'].mode()`

Estadística Descriptiva

Herramientas y medidas para resumir y presentar los datos:

- **Medidas de Dispersión (o Variabilidad):** Indican qué tan dispersos o variados están los datos alrededor del centro.
 - **Rango:** La diferencia entre el valor máximo y el valor mínimo del conjunto de datos.
`rango_matematicas = df['Matematicas'].max() - df['Matematicas'].min()`
 - **Varianza:** Mide la dispersión promedio de los puntos de datos con respecto a la media. Se calcula como el promedio de los cuadrados de las diferencias de cada valor con respecto a la media.
`varianza_matematicas = df['Matematicas'].var()`
 - **Desviación Estándar:** La raíz cuadrada de la varianza. Es la medida de dispersión más común y se interpreta en las mismas unidades que los datos originales.
`std_matematicas = df['Matematicas'].std()`

Estadística Descriptiva

- **Medidas de Posición:** Dividen el conjunto de datos en partes iguales.
 - **Cuartiles:** dividen un conjunto de datos ordenado en cuatro partes iguales, o "cuartos", cada uno conteniendo aproximadamente el 25% de las observaciones.
 - **Q1 (Primer Cuartil o Percentil 25):** Es el valor por debajo del cual cae el 25% más bajo de los datos.
 - **Q2 (Segundo Cuartil o Percentil 50):** Es el valor por debajo del cual cae el 50% más bajo de los datos (o el 50% más alto de los datos cae por encima de él).
 - **Q3 (Tercer Cuartil o Percentil 75):** Es el valor por debajo del cual cae el 75% más bajo de los datos (o el 25% más alto de los datos cae por encima de él).

Estadística Descriptiva

- **Medidas de Posición:** Dividen el conjunto de datos en partes iguales.
 - **Cuartiles:** dividen un conjunto de datos ordenado en cuatro partes iguales, o "cuartos", cada uno conteniendo aproximadamente el 25% de las observaciones.
 - **Utilidad:**
 - **Identificar "Outliers" (Valores atípicos):** Son la base para calcular el Rango Intercuartílico ($RIC = Q3 - Q1$). Si un dato está muy alejado de este rango, probablemente sea un error o un caso muy extraño.
 - **Entender la dispersión:** Si la distancia entre $Q1$ y $Q3$ es muy pequeña, tus datos están muy concentrados. Si es grande, hay mucha variabilidad.

Estadística Descriptiva

- **Medidas de Posición:** Dividen el conjunto de datos en partes iguales.
 - **Cuartiles:** dividen un conjunto de datos ordenado en cuatro partes iguales, o "cuartos", cada uno conteniendo aproximadamente el 25% de las observaciones.

Ejemplo:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# La mayoría de los datos estarán entre 65 y 95, aproximadamente
calificaciones_datos = [
    40, 65, 68, 70, 72, 73, 74,
    75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85,
    86, 87, 88, 89, 90, 92, 95
]

# Crear el DataFrame
df_calificaciones = pd.DataFrame({'Calificacion': calificaciones_datos})

print("DataFrame de Calificaciones:")
print(df_calificaciones)
print(f"\nNúmero de observaciones: {len(df_calificaciones)}")
```

	Calificacion
0	40
1	65
2	68
3	70
4	72
5	73
6	74
7	75
8	76
9	77
10	78
11	79
12	80
13	81
14	82
15	83
16	84
17	85
18	86
19	87
20	88
21	89
22	90
23	92
24	95

Estadística Descriptiva

- **Medidas de Posición:** Dividen el conjunto de datos en partes iguales.
 - **Cuartiles:** dividen un conjunto de datos ordenado en cuatro partes iguales, o "cuartos", cada uno conteniendo aproximadamente el 25% de las observaciones.

Ejemplo:

```
# --- Calcular Q1, Q3 y el IQR para identificar outliers ---  
Q1 = df_calificaciones['Calificacion'].quantile(0.25)  
Q2 = df_calificaciones['Calificacion'].quantile(0.5)  
Q3 = df_calificaciones['Calificacion'].quantile(0.75)  
  
IQR = Q3 - Q1
```

IQR (Rango Intercuartílico): representa el rango del 50% central de un conjunto de datos.

El 50% central de las calificaciones tiene un rango de 12 puntos.

Q1 (25%): 74.00
Q2 (50%): 80.00
Q3 (75%): 86.00
IQR: 12.00

	Calificacion
0	40
1	65
2	68
3	70
4	72
5	73
6	74
7	75
8	76
9	77
10	78
11	79
12	80
13	81
14	82
15	83
16	84
17	85
18	86
19	87
20	88
21	89
22	90
23	92
24	95

Estadística Descriptiva

- **Medidas de Posición:** Dividen el conjunto de datos en partes iguales.
 - **Cuartiles:** dividen un conjunto de datos ordenado en cuatro partes iguales, o "cuartos", cada uno conteniendo aproximadamente el 25% de las observaciones.

Ejemplo:

```
lower_bound_outlier = Q1 - 1.5 * IQR #outlier por debajo.
upper_bound_outlier = Q3 + 1.5 * IQR #outlier por encima.

print(f"\nQ1 (25%): {Q1:.2f}")
print(f"Q2 (50%): {Q2:.2f}")
print(f"Q3 (75%): {Q3:.2f}")
print(f"IQR: {IQR:.2f}")
print(f"Límite inferior para outliers: {lower_bound_outlier:.2f}")
print(f"Límite superior para outliers: {upper_bound_outlier:.2f}")

# Identificar outliers
outliers = df_calificaciones[(df_calificaciones['Calificacion'] < lower_bound_outlier)
                              (df_calificaciones['Calificacion'] > upper_bound_outlier)]

print("\nValores considerados outliers según la regla del 1.5 * IQR:")
print(outliers)
```

Límite inferior para outliers: 56.00
Límite superior para outliers: 104.00

Valores considerados outliers según la regla del 1.5 * IQR:

Calificacion
0
40

Estadística Descriptiva

- **Medidas de Posición:** Dividen el conjunto de datos en partes iguales.
 - **Cuartiles:** dividen un conjunto de datos ordenado en cuatro partes iguales, o "cuartos", cada uno conteniendo aproximadamente el 25% de las observaciones. **Ejemplo:**

#outlier por debajo.

$$\begin{aligned}\text{lower_bound_outlier} &= Q1 - 1.5 * IQR \\ &= 74 - 1.5 * 12 \\ &= 74 - 18 \\ &= 56\end{aligned}$$

#outlier por encima.

$$\begin{aligned}\text{upper_bound_outlier} &= Q3 + 1.5 * IQR \\ &= 86 + 1.5 * 12 \\ &= 86 + 18 \\ &= 104\end{aligned}$$

Q1 (25%): 74.00

Q2 (25%): 80.00

Q3 (75%): 86.00

IQR: 12.00

Límite inferior para outliers: 56.00

Límite superior para outliers: 104.00

Valores considerados outliers según la regla del $1.5 * IQR$:

Calificacion
0
40

Nota: el rango entre el lower_bound y el upper_bound abarca aproximadamente el 99.3% de los datos.

Estadística Descriptiva

- **Medidas de Posición:** Dividen el conjunto de datos en partes iguales.
 - **Cuartiles:** dividen un conjunto de datos ordenado en cuatro partes iguales, o "cuartos", cada uno conteniendo aproximadamente el 25% de las observaciones.

Ejemplo:

```
# --- Visualizar con un Box Plot para confirmar ---  
plt.figure(figsize=(8, 6))  
sns.boxplot(y=df_calificaciones['Calificacion'])  
plt.title('Box Plot de Calificaciones con Outlier Inferior')  
plt.ylabel('Calificación')  
plt.grid(axis='y', linestyle='--', alpha=0.7)  
plt.show()
```

Q1 (25%): 74.00

Q2 (25%): 80.00

Q3 (75%): 86.00

IQR: 12.00

Límite inferior para outliers: 56.00

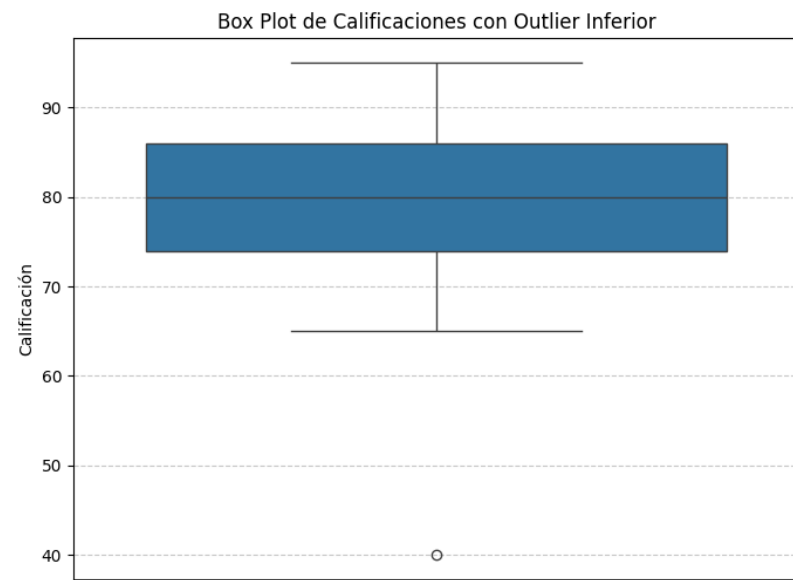
Límite superior para outliers: 104.00

Valores considerados outliers según la regla del $1.5 * IQR$:

Calificación

0

40



Estadística Descriptiva

- **Medidas de Posición:** Dividen el conjunto de datos en partes iguales.
 - **Cuartiles:** dividen un conjunto de datos ordenado en cuatro partes iguales, o "cuartos", cada uno conteniendo aproximadamente el 25% de las observaciones.

Ejemplo:

```
calificaciones_datos = [  
    40, 65, 68, 70, 72, 73, 74,  
    75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85,  
    86, 87, 88, 89, 90, 92, 95  
]
```

Q1 (25%): 74.00

Q2 (25%): 80.00

Q3 (75%): 86.00

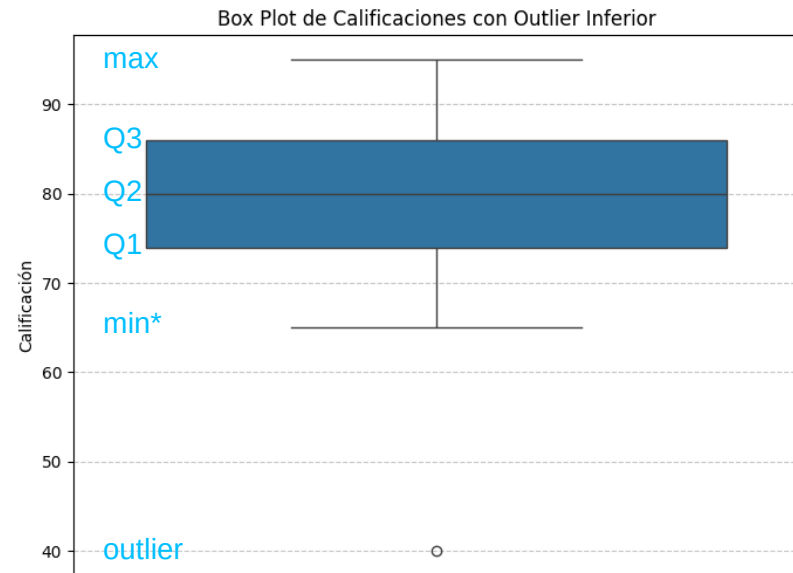
IQR: 12.00

Límite inferior para outliers: 56.00

Límite superior para outliers: 104.00

Valores considerados outliers según la regla del $1.5 * IQR$:

Calificación
0
40



min*: el mínimo sin considerar el outlier.

Estadística Descriptiva

- **Recomendación sobre los outliers:**
 - **Se queda:** Si el dato es real, aunque sea raro (ej. la fortuna de Bill Gates en un estudio de riqueza).
 - **Se va:** Si el dato es un error de medición, un error de carga o no pertenece al grupo de estudio.
 - Antes de borrar un outlier, lo ideal es documentarlo. En lugar de simplemente borrar la fila, marcarla como "inválida" para que otros sepan por qué no se incluyó en el cálculo final.

Estadística Descriptiva

- **Recomendación sobre la dispersión:**

- Usar el IQR_{ratio} :

$$IQR_{ratio} = \frac{Q_3 - Q_1}{Máx - Mín}$$

- $IQR_{ratio} < 0.05$ (menor al 5%): Columna casi plana o dominada por un solo valor. No tiene casi información para el modelo. Se recomienda eliminar esa columna.
- IQR_{ratio} entre 0.25 y 0.60 (25% a 60%): dispersión saludable, el centro abarca una porción equilibrada.
- $IQR_{ratio} > 0.9$ (mayor al 90%): mucha dispersión que puede alterar el modelo predictivo. Aplicar transformación logarítmica o RobusScaler.

Estadística Descriptiva

- **Recomendación sobre la dispersión:**

- Usar el IQRratio:

$$IQR_{ratio} = \frac{Q_3 - Q_1}{Máx - Mín}$$

- $IQR_{ratio} > 0.9$ (mayor al 90%): mucha dispersión que puede alterar el modelo predictivo.
 - Si no hay outliers (dispersión amplia) → MinMaxScaler o StandardScaler
 - Si hay asimetría → transformación logarítmica
 - Si hay muchos outliers → RobustScaler
- Súper tip: si aplicamos un modelo basado en árboles (RandomForests), no es necesario escalar.

Estadística Descriptiva

- **Medidas de Posición:** Dividen el conjunto de datos en partes iguales.
 - **Percentiles:** dividen un conjunto de datos ordenado en 100 partes iguales.
 - Hay 100 percentiles ($P_1, P_2, \dots, P_{100}$).
 - El k-ésimo percentil (P_k) es el valor por debajo del cual se encuentra el k% **de las observaciones** de los datos ordenados.
 - **Ejemplo:**
 - Si un estudiante obtiene una calificación en el percentil 95 en un examen → obtuvo una calificación mejor que el 95% de los demás estudiantes.

Estadística Descriptiva

- **Medidas de Posición:** Dividen el conjunto de datos en partes iguales.
 - **Percentiles:** dividen un conjunto de datos ordenado en 100 partes iguales.
 - **Ejemplo:**

Dada esta lista: [60, 65, 70, 72, 75, 80, 85, 88, 90, 92, 95] → la lista debe estar ordenada

$$L = \frac{k}{100} \times N$$

N: Número de elementos → 11

k: percentil → 70

L: elemento de la lista que cumple con k.

$L = (70/100) \times 11 = 7.7 \rightarrow 8 \rightarrow$ el elemento 8 (88) es percentil 70.

Estadística Descriptiva

- **Medidas de Posición:** Dividen el conjunto de datos en partes iguales.
 - **Percentiles:** dividen un conjunto de datos ordenado en 100 partes iguales.

```
import pandas as pd
import numpy as np

# Datos de ejemplo
calificaciones = [60, 65, 70, 72, 75, 80, 85, 88, 90, 92, 95]
df_calificaciones = pd.DataFrame({'Calificacion': calificaciones})

print("DataFrame Original de Calificaciones:")
print(df_calificaciones)
print(f"Número total de datos: {len(df_calificaciones)}")

# --- Usando Pandas ---
# El método .quantile() espera la fracción (0.0 a 1.0)
# Calcular el percentil 70 (0.70)
percentil_70_pandas = df_calificaciones['Calificacion'].quantile(0.70)
print(f"\nPercentil 70 (Pandas): {percentil_70_pandas:.2f}")

# Calcular múltiples percentiles a la vez (ej. P10, P50 (mediana), P90)
multi_percentiles_pandas = df_calificaciones['Calificacion'].quantile([0.10, 0.50, 0.90])
print("\nMúltiples percentiles (Pandas):")
print(multi_percentiles_pandas)
```

Estadística Descriptiva

- **Medidas de Posición:** Dividen el conjunto de datos en partes iguales.
 - **Percentiles:** dividen un conjunto de datos ordenado en 100 partes iguales.

- **Ejemplo:**

```
# --- Usando NumPy ---  
# El método np.percentile() espera el percentil como un número entero (0 a 100)  
percentil_70_numpy = np.percentile(calificaciones, 70)  
print(f"\nPercentil 70 (NumPy): {percentil_70_numpy:.2f}")  
  
# Calcular múltiples percentiles con NumPy  
multi_percentiles_numpy = np.percentile(calificaciones, [10, 50, 90])  
print("\nMúltiples percentiles (NumPy):")  
print(multi_percentiles_numpy)
```

DataFrame Original de Calificaciones:

Calificacion

0	60
1	65
2	70
3	72
4	75
5	80
6	85
7	88
8	90
9	92
10	95

Número total de datos: 11

Percentil 70 (Pandas): 88.00

Múltiples percentiles (Pandas):

0.1	65.0
0.5	80.0
0.9	92.0

Name: Calificacion, dtype: float64

Percentil 70 (NumPy): 88.00

Múltiples percentiles (NumPy):

[65. 80. 92.]

Estadística Descriptiva

- **Gráfico de barras:** representa datos categóricos (o discretos) mediante barras rectangulares.
 - **Eje de Categorías (Generalmente X):** muestra las categorías discretas que estás comparando.
Ejemplo: Tipos de productos, géneros, meses del año, marcas de automóviles, etc.
 - **Eje de Valor (Generalmente Y):** representa la medida para cada categoría.
Ejemplo:
 - **Frecuencia (conteo):** Cuántas veces aparece cada categoría.
 - **Suma:** El total de alguna variable numérica para cada categoría
 - **Promedio:** El promedio de alguna variable numérica por categoría.
 - **Porcentaje/Proporción:** El porcentaje que cada categoría representa del total.

Estadística Descriptiva

- **Gráfico de barras:** representa datos categóricos (o discretos) mediante barras rectangulares.

Ejemplo:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np # Lo usaremos para generar datos de ejemplo

# 1. Crear un DataFrame de ejemplo
# Simularemos datos de ventas de diferentes productos
data = {
    'Producto': ['Laptop', 'Smartphone', 'Tablet', 'Auriculares', 'Smartwatch', 'Laptop', 'Smartphone', 'Tablet', 'Auriculares', 'Smartwatch'],
    'Ventas_Unidades': [10, 15, 8, 25, 12, 18, 10, 20, 17, 14, 22],
    'Precio_Unitario': [1200, 800, 450, 100, 250, 1200, 450, 100, 800, 250, 1200]
}
df_ventas = pd.DataFrame(data)

# Calcular el valor total de ventas por cada transacción
df_ventas['Valor_Venta'] = df_ventas['Ventas_Unidades'] * df_ventas['Precio_Unitario']

print("DataFrame de Ventas Original:")
print(df_ventas)

# 2. Preparar los datos para el gráfico de barras
# Queremos ver el VALOR TOTAL de ventas por cada PRODUCTO.
# Para esto, agrupamos por 'Producto' y sumamos 'Valor_Venta'.
ventas_por_producto = df_ventas.groupby('Producto')['Valor_Venta'].sum().reset_index()

# Opcional: ordenar los productos por el valor total de ventas para una mejor visualización
ventas_por_producto = ventas_por_producto.sort_values(by='Valor_Venta', ascending=False)
```

Estadística Descriptiva

- **Gráfico de barras:** representa datos categóricos (o discretos) mediante barras rectangulares.

Ejemplo:

```
print("\nDataFrame de Ventas Agrupadas por Producto:")
print(ventas_por_producto)

# 3. Crear el Gráfico de Barras
plt.figure(figsize=(10, 6)) # Define el tamaño del gráfico

# Usamos sns.barplot() de Seaborn
# x: La columna categórica que queremos representar en el eje X
# y: La columna numérica que queremos que determine la altura de las barras (sumada por defecto)
# palette: Define la paleta de colores para las barras
sns.barplot(x='Producto', y='Valor_Venta', data=ventas_por_producto, palette='viridis')

# 4. Personalizar el Gráfico (Títulos, Etiquetas, etc.)
plt.title('Valor Total de Ventas por Tipo de Producto', fontsize=16)
plt.xlabel('Tipo de Producto', fontsize=12)
plt.ylabel('Valor Total de Ventas ($)', fontsize=12)

# Rotar etiquetas del eje X si los nombres de productos son largos
plt.xticks(rotation=45, ha='right')

# Añadir una cuadrícula al fondo para facilitar la lectura
plt.grid(axis='y', linestyle='--', alpha=0.7)

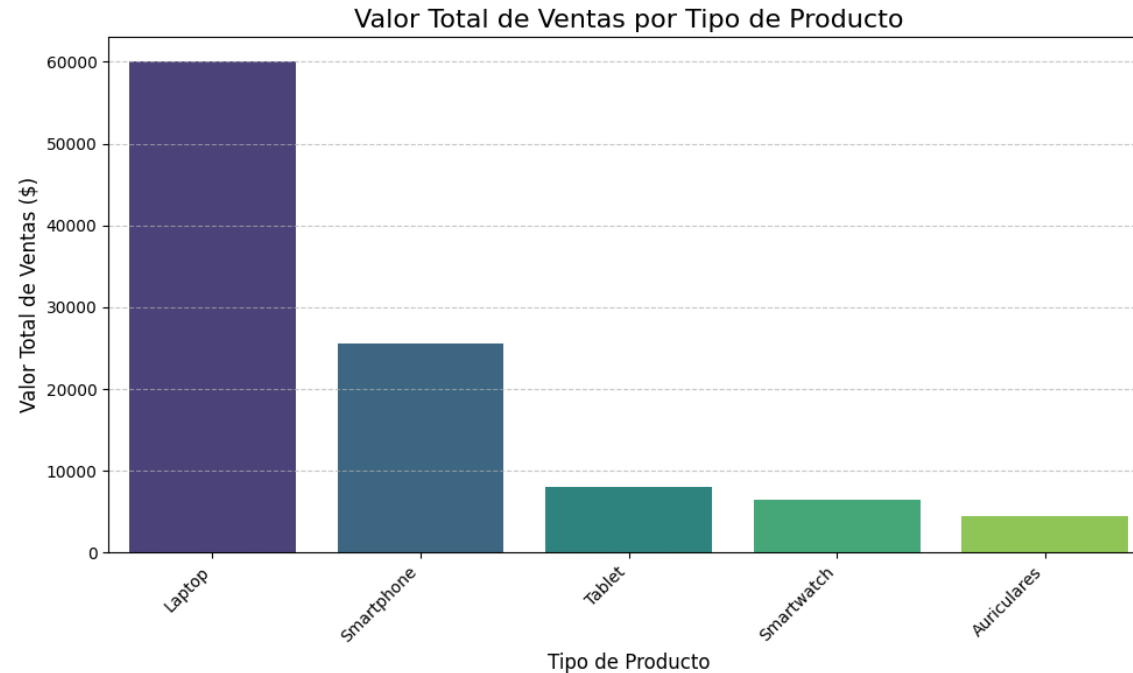
# Ajustar el diseño para asegurar que todo quepa
plt.tight_layout()

# 5. Mostrar el gráfico
plt.show()
```

Estadística Descriptiva

- **Gráfico de barras:** representa datos categóricos (o discretos) mediante barras rectangulares.

Ejemplo:



Estadística Descriptiva

- **Gráfico circulares:** representan la proporción o el porcentaje de cada categoría dentro de un conjunto de datos.

Ejemplo:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns # Incluido por convención, aunque plt.pie es de Matplotlib

# Crear un DataFrame a partir de un diccionario.
# Aquí, los datos ya están en el formato de conteo por categoría
data_frutas = {
    'Fruta': ['Manzana', 'Banana', 'Naranja', 'Uva', 'Pera'],
    'Votos': [30, 25, 15, 20, 10]
}
df_frutas = pd.DataFrame(data_frutas)

print("DataFrame de Frutas Favoritas:")
print(df_frutas)
```

Estadística Descriptiva

- **Gráfico circulares:** representan la proporción o el porcentaje de cada categoría dentro de un conjunto de datos.

Ejemplo:

```
# Los datos ya están listos para el gráfico circular:
# 'Fruta' serán las etiquetas de los segmentos (labels)
# 'Votos' serán los valores que determinan el tamaño de los segmentos (x)

# plt.pie() es la función de Matplotlib para gráficos circulares
plt.pie(
    df_frutas['Votos'],          # Los valores que determinan el tamaño de cada rebanada
    labels=df_frutas['Fruta'],  # Las etiquetas para cada rebanada
    autopct='%1.1f%%',         # Formato para mostrar el porcentaje en cada rebanada (ej. "25.0%")
    startangle=90,              # El ángulo donde comienza la primera rebanada (vertical hacia arriba)
    colors=sns.color_palette('pastel') # Una bonita paleta de colores suaves
)

# 3. Personalizar el Gráfico
plt.title('Distribución de Frutas Favoritas', fontsize=16)
plt.axis('equal') # Asegura que el gráfico sea un círculo perfecto y no una elipse

# 4. Mostrar el gráfico
plt.show()
```

Estadística Descriptiva

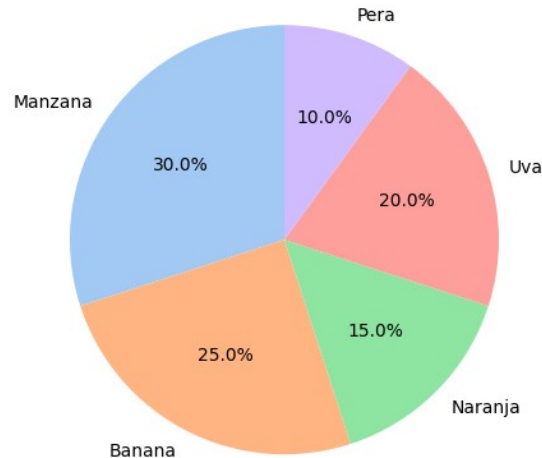
- **Gráfico circulares:** representan la proporción o el porcentaje de cada categoría dentro de un conjunto de datos.

Ejemplo:

DataFrame de Frutas Favoritas:

	Fruta	Votos
0	Manzana	30
1	Banana	25
2	Naranja	15
3	Uva	20
4	Pera	10

Distribución de Frutas Favoritas



Estadística Descriptiva

- **Gráfico circulares:** representan la proporción o el porcentaje de cada categoría dentro de un conjunto de datos.

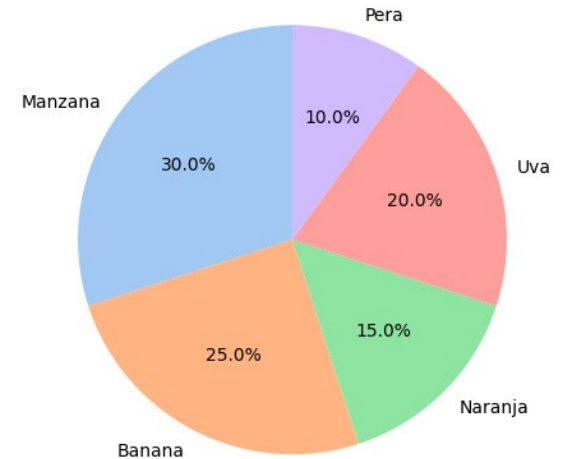
Desventaja:

Se vuelve difícil de comprender si tenemos mas de 6 o 7 categorías.

DataFrame de Frutas Favoritas:

	Fruta	Votos
0	Manzana	30
1	Banana	25
2	Naranja	15
3	Uva	20
4	Pera	10

Distribución de Frutas Favoritas



Estadística Descriptiva

- **Histograma:** es útil para visualizar la distribución de frecuencia de una variable numérica (cuantitativa).
 - En otras palabras, muestra cuántas veces ocurren los valores dentro de ciertos rangos o "intervalos".
 - **Componentes:**
 - **Eje X (Horizontal):** Representa los rangos o intervalos (bins o clases) de la variable numérica que estás analizando. Cada barra del histograma corresponde a uno de estos intervalos.
 - **Ejemplo:** Si estamos graficando edades, los bins podrían ser "0-10", "11-20", "21-30", etc.

Estadística Descriptiva

- **Histograma:** es útil para visualizar la distribución de frecuencia de una variable numérica (cuantitativa).
 - **Componentes:**
 - **Eje Y (Vertical):** Representa la frecuencia (o el conteo) de las observaciones que caen dentro de cada intervalo.

Estadística Descriptiva

- **Histograma:** es útil para visualizar la distribución de frecuencia de una variable numérica (cuantitativa).

Ejemplo:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Generar un DataFrame de salarios anuales (reutilizando el ejemplo anterior)
np.random.seed(42) # Para reproducibilidad

salarios_bajos = np.random.normal(loc=40000, scale=8000, size=900)
salarios_altos = np.random.normal(loc=120000, scale=25000, size=100)
salarios_super_altos = np.random.normal(loc=300000, scale=50000, size=10)

salarios = np.concatenate([salarios_bajos, salarios_altos, salarios_super_altos])

#Evitar valores negativos o irrealmente bajos:
salarios = np.maximum(salarios, 15000)

df_salarios = pd.DataFrame({'Salario_Anual': salarios})

print("DataFrame de Salarios (primeras 5 filas):")
print(df_salarios.head())
print(f"\nNúmero total de empleados: {len(df_salarios)}")

# Calcular la media y la mediana para visualizarlas en el histograma
media_salario = df_salarios['Salario_Anual'].mean()
mediana_salario = df_salarios['Salario_Anual'].median()
```

Estadística Descriptiva

- **Histograma:** es útil para visualizar la distribución de frecuencia de una variable numérica (cuantitativa).

Ejemplo:

```
# data: el DataFrame
# x: la columna que queremos graficar
# bins: Número de intervalos o una secuencia de bordes de bin
# color: Color de las barras
#sns.histplot(data=df_salarios, x='Salario_Anual', kde=True, bins=50, color='skyblue')
sns.histplot(data=df_salarios, x='Salario_Anual', bins=50, color='skyblue')

# Añadir líneas para la media y la mediana (opcional, pero útil para EDA)
plt.axvline(media_salario, color='red', linestyle='--', label=f'Media: ${media_salario:,.0f}')
plt.axvline(mediana_salario, color='green', linestyle='-', label=f'Mediana: ${mediana_salario:,.0f}')

# Añadir título y etiquetas a los ejes
plt.title('Distribución de Salarios Anuales (Histograma)', fontsize=16)
plt.xlabel('Salario Anual ($)', fontsize=12)
plt.ylabel('Frecuencia de Empleados', fontsize=12)

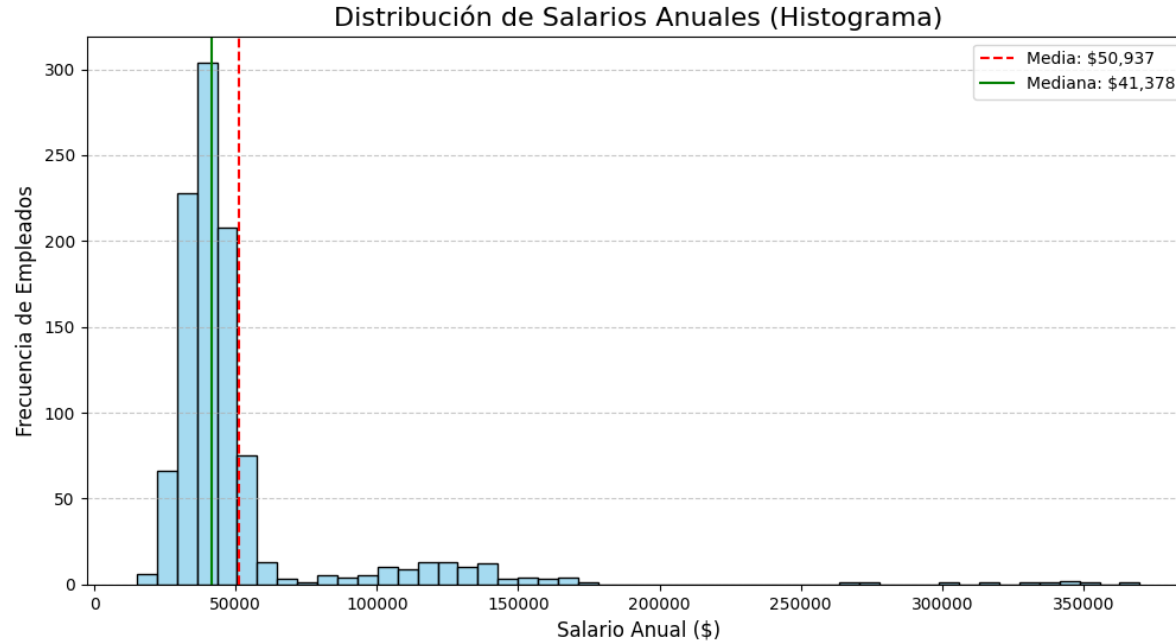
# Añadir leyenda y cuadrícula
plt.legend()
plt.grid(axis='y', linestyle='--', alpha=0.7)

# Mostrar el gráfico
plt.show()
```

Estadística Descriptiva

- **Histograma:** es útil para visualizar la distribución de frecuencia de una variable numérica (cuantitativa).

Ejemplo:



Estadística Descriptiva

- **Gráficos de Dispersión (Scatter Plots):** muestran visualmente la relación entre dos variables numéricas.

Ejemplo: en una empresa de software queremos ver si hay una relación entre el número de horas de desarrollo que un equipo invierte en un proyecto y el número de errores (bugs) encontrados en el software final.

Estadística Descriptiva

- **Gráficos de Dispersión (Scatter Plots):** muestran visualmente la relación entre dos variables numéricas.

Ejemplo:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Crear un DataFrame de ejemplo
# Simularemos datos de proyectos de software
np.random.seed(29) # Para reproducibilidad

num_proyectos = 120

# Horas de desarrollo por proyecto (simulamos una distribución normal)
horas_desarrollo = np.random.normal(loc=1500, scale=400, size=num_proyectos).round(0)
# Asegurarnos de que las horas no sean negativas o irrealmente bajas
horas_desarrollo = np.maximum(horas_desarrollo, 500)

# Número de errores (bugs)
# Modelamos una relación negativa con las horas, pero con ruido y cierta base de errores
# Cuantas más horas, menos errores base
errores = 100 - (horas_desarrollo / 50) + np.random.normal(loc=0, scale=15, size=num_proyectos)
# Asegurarnos de que los errores no sean negativos
errores = np.maximum(errores, 5).round(0) # Mínimo 5 errores, redondeado

df_proyectos = pd.DataFrame({
    'Horas_Desarrollo': horas_desarrollo,
    'Errores_Encontrados': errores
})

print("DataFrame de Proyectos (primeras 5 filas):")
print(df_proyectos.head())
print(f"\nNúmero total de proyectos: {len(df_proyectos)}")
```

Estadística Descriptiva

- **Gráficos de Dispersión (Scatter Plots):** muestran visualmente la relación entre dos variables numéricas.

Ejemplo:

```
# 2. Calcular la Correlación (opcional, pero útil para scatter plots)
correlacion = df_proyectos['Horas_Desarrollo'].corr(df_proyectos['Errores_Encontrados'])
print(f"\nCoeficiente de Correlación entre Horas de Desarrollo y Errores: {correlacion:.3f}")

# 3. Crear el Gráfico de Dispersión
plt.figure(figsize=(10, 7)) # Definimos el tamaño del gráfico

# Usamos sns.scatterplot()
# x: Horas de Desarrollo (variable independiente)
# y: Errores Encontrados (variable dependiente)
# data: El DataFrame con nuestros datos
# hue: Podemos añadir otra dimensión, por ejemplo, colorear por un rango de errores (esto es solo para visualización)
#sns.scatterplot(x='Horas_Desarrollo', y='Errores_Encontrados', data=df_proyectos,
#               hue=pd.cut(df_proyectos['Errores_Encontrados'], bins=3, labels=['Bajo', 'Medio', 'Alto']),
#               size='Horas_Desarrollo', sizes=(50, 500), alpha=0.7, palette='coolwarm')
# No usaremos hue/size para mantenerlo más sencillo en el gráfico, solo x, y y data.
sns.scatterplot(x='Horas_Desarrollo', y='Errores_Encontrados', data=df_proyectos,
               color='teal', alpha=0.7, s=100) # s es el tamaño del punto
```

Estadística Descriptiva

- **Gráficos de Dispersión (Scatter Plots):** muestran visualmente la relación o correlación entre dos variables numéricas.

Ejemplo:

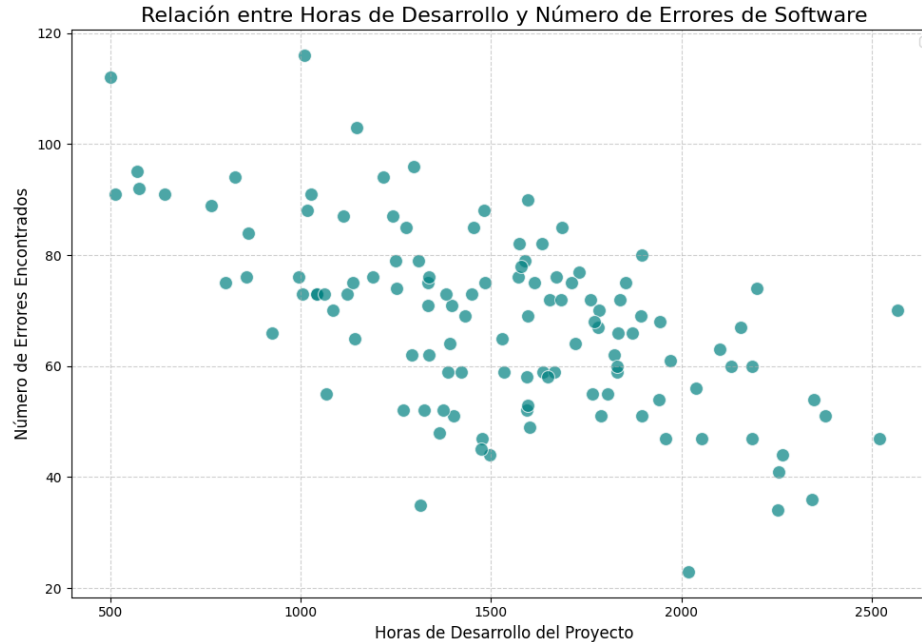
```
# 4. Personalizar el Gráfico
plt.title('Relación entre Horas de Desarrollo y Número de Errores de Software', fontsize=16)
plt.xlabel('Horas de Desarrollo del Proyecto', fontsize=12)
plt.ylabel('Número de Errores Encontrados', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend() # Muestra la leyenda de la línea de tendencia

# 5. Mostrar el gráfico
plt.tight_layout() # Ajusta el diseño para evitar recortes
plt.show()
```

Estadística Descriptiva

- **Gráficos de Dispersión (Scatter Plots):** muestran visualmente la relación o correlación entre dos variables numéricas.

Ejemplo:



Coefficiente de Correlación entre
Horas de Desarrollo y Errores: -0.591

Estadística Descriptiva

- **Mapas de calor:** muestran valores individuales de una matriz representados como colores.
 - Las filas y columnas representan diferentes categorías o variables, y las celdas interiores contienen valores numéricos.

Estadística Descriptiva

- **Mapas de calor:** muestran valores individuales de una matriz representados como colores.

Ejemplo:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Crear un DataFrame de ejemplo muy simple
# Este DataFrame ya está en el formato de "matriz" que necesita un mapa de calor.
# Las filas son los días, las columnas los productos, y los valores son las ventas.
data_ventas_diarias = {
    'Laptops': [5, 2, 8, 4, 6],
    'Smartphones': [10, 7, 12, 9, 11],
    'Tablets': [3, 1, 5, 2, 4]
}

# Creamos el DataFrame y le asignamos los días de la semana como índice
df_ventas_por_dia = pd.DataFrame(data_ventas_diarias,
                                index=['Lunes', 'Martes', 'Miércoles', 'Jueves', 'Viernes'])

print("DataFrame de Ventas por Día y Producto:")
print(df_ventas_por_dia)
```

Estadística Descriptiva

- **Mapas de calor:** muestran valores individuales de una matriz representados como colores.

Ejemplo:

```
# 2. Crear el Mapa de Calor
plt.figure(figsize=(8, 6)) # Ajustamos el tamaño para una buena visualización

# sns.heatmap() es la función clave
# data: El DataFrame que ya está en el formato de matriz deseado
# annot=True: Muestra los valores numéricos en cada celda
# cmap: La paleta de colores. 'YlGnBu' (Yellow-Green-Blue) es buena para valores ascendentes.
# fmt='g': Formato de los números (general, sin decimales para conteos enteros)
# linewidths=.5: Añade un pequeño borde blanco entre las celdas
# cbar_kws={'label': 'Cantidad de Ventas'}: Etiqueta para la barra de color lateral
sns.heatmap(df_ventas_por_dia,
            annot=True,
            cmap='YlGnBu',
            fmt='g',
            linewidths=.5,
            cbar_kws={'label': 'Cantidad de Ventas'})

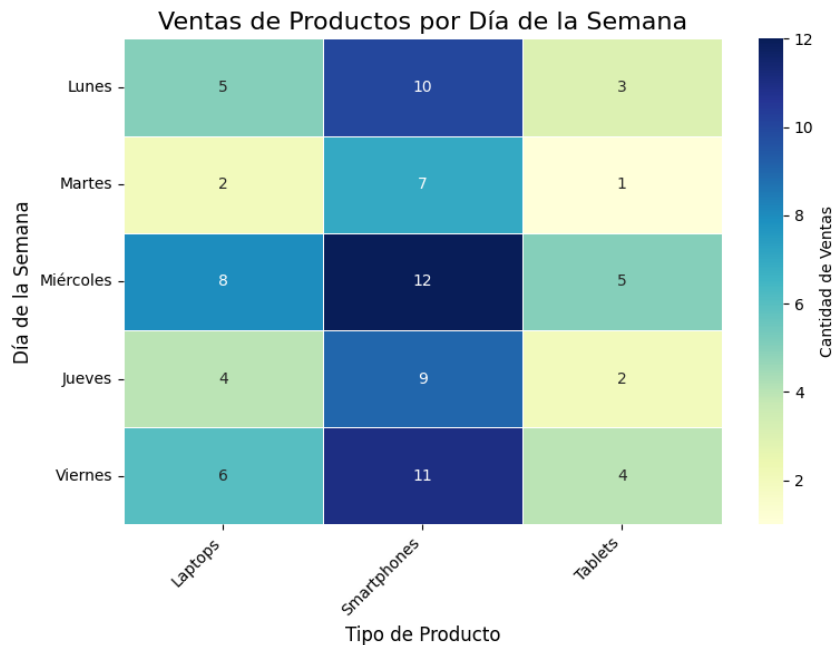
# 3. Personalizar el Gráfico
plt.title('Ventas de Productos por Día de la Semana', fontsize=16)
plt.xlabel('Tipo de Producto', fontsize=12)
plt.ylabel('Día de la Semana', fontsize=12)
plt.xticks(rotation=45, ha='right') # Opcional: rotar etiquetas si se superponen
plt.yticks(rotation=0)
plt.tight_layout() # Ajusta el diseño para evitar que los elementos se superpongan

# 4. Mostrar el gráfico
plt.show()
```

Estadística Descriptiva

- **Mapas de calor:** muestran valores individuales de una matriz representados como colores.

Ejemplo:



DataFrame de Ventas por Día y Producto:

	Laptops	Smartphones	Tablets
Lunes	5	10	3
Martes	2	7	1
Miércoles	8	12	5
Jueves	4	9	2
Viernes	6	11	4

Estadística Descriptiva

- **Gráficos de radar:** muestran valores individuales de una matriz representados como colores.
 - Las filas y columnas representan diferentes categorías o variables, y las celdas interiores contienen valores numéricos.

Estadística Descriptiva

- **Gráficos de radar:** muestran valores individuales de una matriz representados como colores.

Ejemplo:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. Crear el DataFrame de ejemplo
data = {
    'Habilidad': ['Python', 'SQL', 'Cloud', 'Comunicación', 'Resolución de Problemas', 'Trabajo en Equipo'],
    'Equipo Alfa': [8, 7, 9, 6, 7, 8],
    'Equipo Beta': [7, 9, 6, 8, 8, 7],
    'Equipo Gamma': [6, 6, 7, 9, 9, 9]
}

df_habilidades = pd.DataFrame(data)

# Establecer la columna 'Habilidad' como índice para facilitar la transposición
df_habilidades = df_habilidades.set_index('Habilidad')

print("DataFrame de Habilidades de los Equipos:")
print(df_habilidades)
```

Estadística Descriptiva

- **Gráficos de radar:** muestran valores individuales de una matriz representados como colores.

Ejemplo:

```
# 2. Preparar los datos para el gráfico de araña
# Las "Habilidades" serán nuestros ejes radiales (variables).
habilidades = df_habilidades.index.tolist()
num_habilidades = len(habilidades)

# Calcular el ángulo para cada eje radial
# Sumamos el primer ángulo al final para cerrar el polígono en el gráfico
angles = np.linspace(0, 2 * np.pi, num_habilidades, endpoint=False).tolist()
angles += angles[:1] # Esto es clave para que el polígono se cierre

# Extraer los datos para cada equipo.
valores_alfa = df_habilidades['Equipo Alfa'].tolist()
valores_alfa += valores_alfa[:1]

valores_beta = df_habilidades['Equipo Beta'].tolist()
valores_beta += valores_beta[:1]

valores_gamma = df_habilidades['Equipo Gamma'].tolist()
valores_gamma += valores_gamma[:1]
```

Estadística Descriptiva

- **Gráficos de radar:** muestran valores individuales de una matriz representados como colores.

Ejemplo:

```
# 3. Crear el Gráfico de Araña
plt.figure(figsize=(9, 9)) # Ajusta el tamaño de la figura para un buen círculo
ax = plt.subplot(111, polar=True) # Creamos un subplot con proyección polar

# Dibujar cada polígono para cada equipo
ax.plot(angles, valores_alfa, 'o-', linewidth=2, linestyle='solid', label='Equipo Alfa', color='blue')
ax.fill(angles, valores_alfa, color='blue', alpha=0.25) # Rellena el área para mejor visualización

ax.plot(angles, valores_beta, 'o-', linewidth=2, linestyle='solid', label='Equipo Beta', color='green')
ax.fill(angles, valores_beta, color='green', alpha=0.25)

ax.plot(angles, valores_gamma, 'o-', linewidth=2, linestyle='solid', label='Equipo Gamma', color='red')
ax.fill(angles, valores_gamma, color='red', alpha=0.25)
```

Estadística Descriptiva

- **Gráficos de radar:** muestran valores individuales de una matriz representados como colores.

Ejemplo:

```
# 4. Personalizar el Gráfico
# Establecer los ticks para los ejes radiales (la escala)
ax.set_rticks(np.arange(0, 11, 2)) # Ticks en 0, 2, 4, 6, 8, 10
ax.set_rlabel_position(0) # Posición de las etiquetas de la escala
ax.set_rlim(0, 10) # Rango de la escala de 0 a 10

# Establecer las etiquetas de las habilidades en los ejes
ax.set_xticks(angels[:-1]) # Establecer la posición de las etiquetas
ax.set_xticklabels(habilidades, fontsize=12) # Asignar los nombres de las habilidades

# Añadir título y leyenda
plt.title('Comparación de Habilidades de Equipos de Desarrollo', va='bottom', fontsize=16)
plt.legend(loc='upper right', bbox_to_anchor=(1.3, 1.1)) # Posicionar la leyenda fuera del gráfico

# Mostrar el gráfico
plt.tight_layout()
plt.show()
```

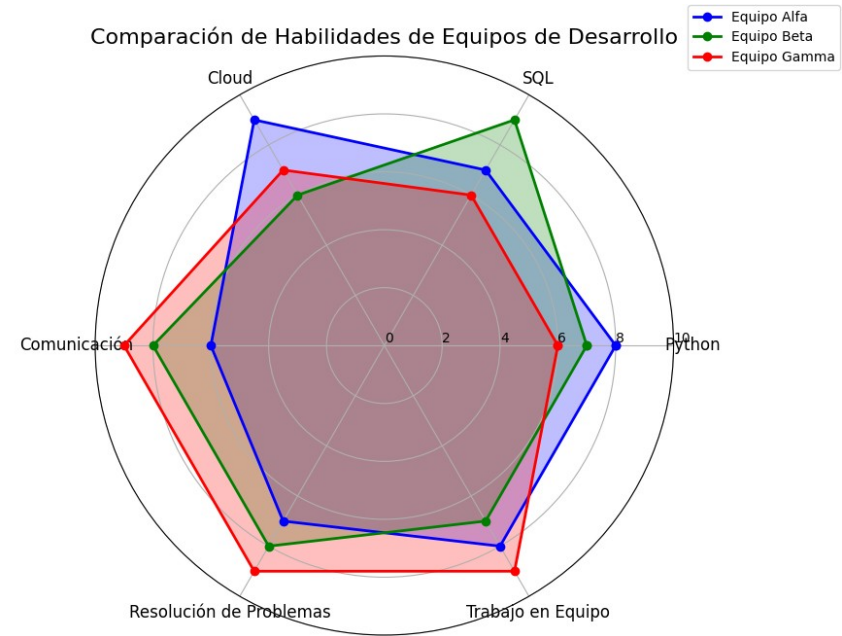
Estadística Descriptiva

- **Gráficos de radar:** muestran valores individuales de una matriz representados como colores.

Ejemplo:

DataFrame de Habilidades de los Equipos:

	Equipo Alfa	Equipo Beta	Equipo Gamma
Habilidad			
Python	8	7	6
SQL	7	9	6
Cloud	9	6	7
Comunicación	6	8	9
Resolución de Problemas	7	8	9
Trabajo en Equipo	8	7	9



Estadística Descriptiva

- **Gráficos de Series de Tiempo:** Son útiles para visualizar cómo una o más variables cambian y evolucionan a lo largo del tiempo.
- **Características:**
 - **Eje X (Horizontal):** Siempre representa el tiempo. Esto puede ser en diferentes unidades: segundos, minutos, horas, días, semanas, meses, años, etc. Es crucial que el tiempo esté ordenado cronológicamente.
 - **Eje Y (Vertical):** Representa los valores de la variable(s) que se están midiendo en cada punto del tiempo. Esta variable es siempre numérica.
 - **Puntos Conectados:** Los puntos de datos se conectan con líneas. Esta conexión enfatiza la progresión continua y la relación de un valor con el siguiente en la secuencia temporal. Si hay múltiples series de tiempo, cada una tendrá su propia línea.

Estadística Descriptiva

- **Gráficos de Series de Tiempo:** Son útiles para visualizar cómo una o más variables cambian y evolucionan a lo largo del tiempo.

Ejemplo:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from datetime import datetime, timedelta

# 1. Crear un DataFrame de ejemplo con datos de series de tiempo
np.random.seed(42) # Para reproducibilidad

# Generar un rango de fechas
start_date = datetime(2024, 1, 1)
dates = [start_date + timedelta(days=i) for i in range(180)] # 180 días (aprox. 6 meses)

# Simular precios de acciones con una tendencia al alza y algo de ruido
precio_base = 150.0
tendencia = np.linspace(0, 50, len(dates)) # Tendencia creciente a lo largo del tiempo
ruido = np.random.normal(0, 5, len(dates)) # Fluctuaciones diarias
precios = precio_base + tendencia + ruido
precios = np.maximum(precios, 140).round(2) # Asegurar que no bajen mucho, redondear a 2 decimales

# Simular volumen de operaciones (con más volumen en precios más altos, quizás)
volumen_base = 100000
volumen_variacion = np.random.normal(0, 20000, len(dates))
volumen = (volumen_base + volumen_variacion + (precios * 100)).round(0)
volumen = np.maximum(volumen, 50000) # Asegurar un volumen mínimo
```

Estadística Descriptiva

- **Gráficos de Series de Tiempo:** Son útiles para visualizar cómo una o más variables cambian y evolucionan a lo largo del tiempo.

Ejemplo:

```
df_acciones = pd.DataFrame({
    'Fecha': dates,
    'Precio_Cierre': precios,
    'Volumen': volumen
})

# Establecer la columna 'Fecha' como el índice del DataFrame. Esto es crucial para series de tiempo en Pandas.
df_acciones = df_acciones.set_index('Fecha')

print("DataFrame de Precios de Acciones (primeras 5 filas):")
print(df_acciones.head())
print(f"\nNúmero total de días: {len(df_acciones)}")

# 2. Crear el Gráfico de Series de Tiempo
plt.figure(figsize=(14, 7)) # Tamaño de la figura para una buena visualización

# Gráfico para Precio de Cierre
plt.plot(df_acciones.index, df_acciones['Precio_Cierre'], label='Precio de Cierre ($)', color='blue', linewidth=2)

# Gráfico para Volumen (usando un segundo eje Y para escalas diferentes)
# Esto es común cuando las unidades de las variables son muy distintas.
ax2 = plt.gca().twinx() # Crea un segundo eje Y que comparte el mismo eje X
ax2.plot(df_acciones.index, df_acciones['Volumen'], label='Volumen Operado', color='green', linestyle='--', alpha=0.6)
```

Estadística Descriptiva

- **Gráficos de Series de Tiempo:** Son útiles para visualizar cómo una o más variables cambian y evolucionan a lo largo del tiempo.

Ejemplo:

```
# 3. Personalizar el Gráfico
plt.title('Evolución del Precio de Cierre y Volumen de Acciones a lo Largo del Tiempo', fontsize=18)
plt.xlabel('Fecha', fontsize=14)

# Etiquetas para los dos ejes Y
plt.ylabel('Precio de Cierre ($)', fontsize=14, color='blue')
ax2.set_ylabel('Volumen Operado', fontsize=14, color='green')

# Configurar leyendas para ambos ejes
plt.legend(loc='upper left')
ax2.legend(loc='upper right')

plt.grid(True, linestyle=':', alpha=0.7) # Añadir una cuadrícula

# Formatear el eje X para mostrar las fechas de forma legible
plt.xticks(rotation=45, ha='right')

# 4. Mostrar el gráfico
plt.tight_layout() # Ajusta el diseño para evitar superposiciones
plt.show()
```

Estadística Descriptiva

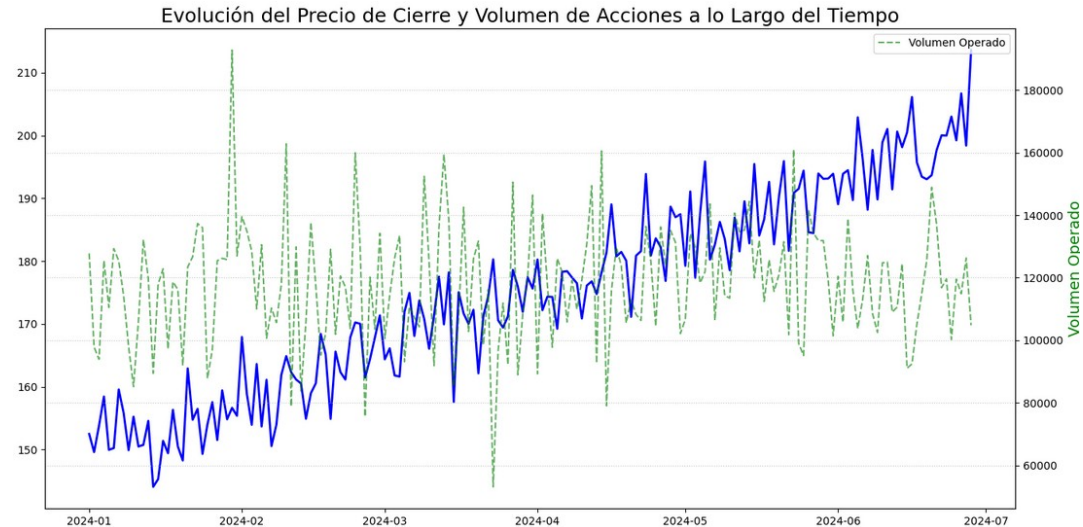
- **Gráficos de Series de Tiempo:** Son útiles para visualizar cómo una o más variables cambian y evolucionan a lo largo del tiempo.

Ejemplo:

DataFrame de Precios de Acciones (primeras 5 filas):

Fecha	Precio_Cierre	Volumen
2024-01-01	152.48	127761.0
2024-01-02	149.59	97816.0
2024-01-03	153.80	93962.0
2024-01-04	158.45	125494.0
2024-01-05	149.95	110526.0

Número total de días: 180



EDA automático

- **Definición:** son herramientas que realizan EDA de manera automática.
- **Algunas opciones:**
 - **YData Profiling:** Es el estándar de la industria. Genera un reporte interactivo en HTML que incluye:
 - Estadísticas descriptivas: Media, mediana, valores faltantes.
 - Alertas: avisa si hay variables muy correlacionadas o con muchos ceros.
 - Correlaciones: Matrices de Pearson, Spearman, etc.

EDA automático

- **Definición:** son herramientas que realizan EDA de manera automática.
- **Algunas opciones:**
 - **Sweetviz:** Es muy visual y permite comparar dos datasets (por ejemplo, Train vs Test) o subconjuntos de datos (Hombres vs Mujeres) de forma inmediata.
 - **D-Tale:** abre una interfaz interactiva en tu navegador que se siente como un Excel supervitaminado. Se pueden filtrar datos, crear gráficos personalizados y exportar el código de Python de lo que se va haciendo.

EDA automático

- **Ejemplo de YData Profiling:**

```
!pip install ydata-profiling
import pandas as pd
from ydata_profiling import ProfileReport

# 1. Cargar los datos
# Usaremos un dataset de ejemplo de internet
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

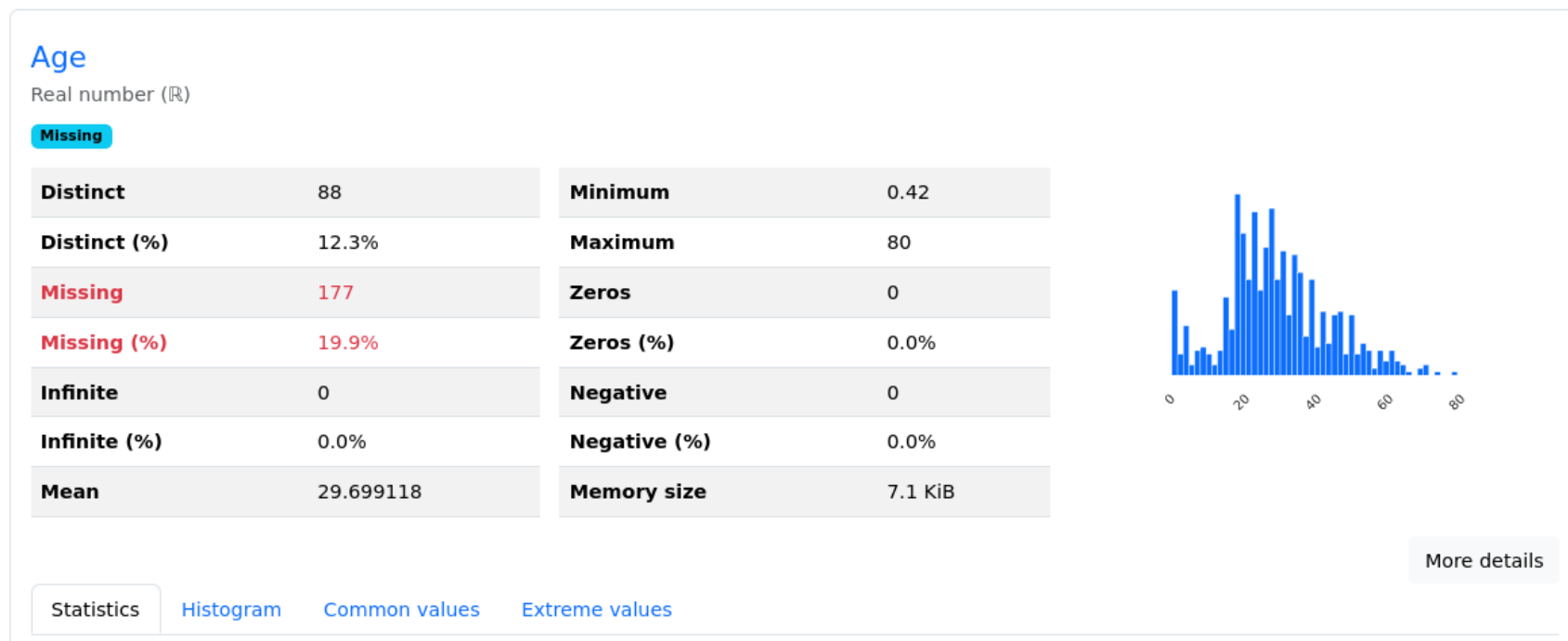
# 2. Generar el reporte
profile = ProfileReport(df, title="Reporte de EDA Automático - Titanic")

# 3. Ver el reporte
profile.to_notebook_iframe()

# 0 si prefieres guardarlo como un archivo independiente:
profile.to_file("mi_analisis_exploratorio.html")
```

EDA automático

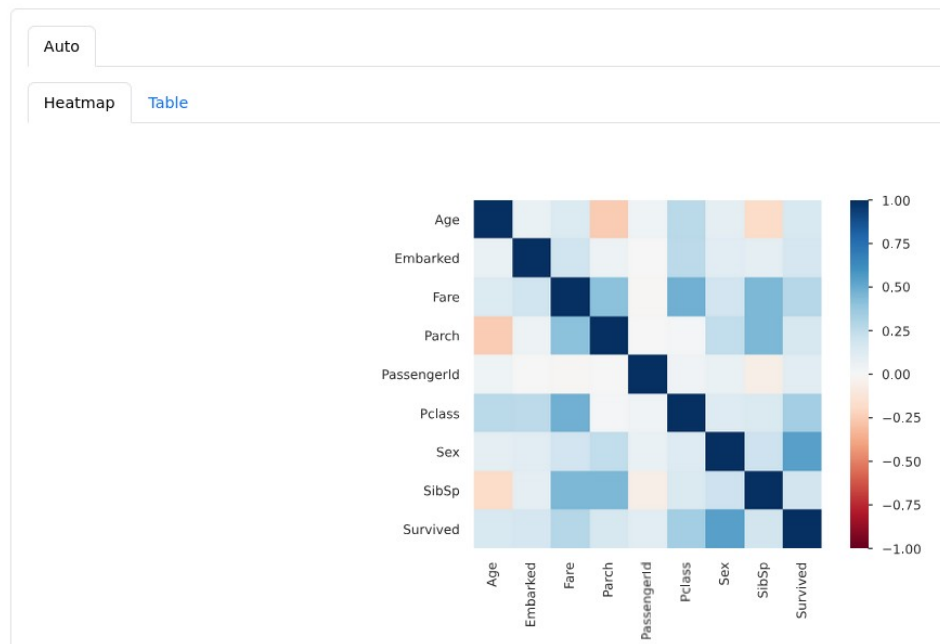
- Ejemplo de YData Profiling:



EDA automático

- **Ejemplo de YData Profiling:**

Correlations



EDA automático

- Ejemplo de YData Profiling:

Sample

First rows

Last rows

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38.0	1	0	PC 17599	71.2833
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708